

Design Specification

SmartSociety — Web-Based Housing Society Management System

Companion to Problem Definition

Date August 2026

1 Architecture Overview

The system is a three-tier web application: a React single-page application talking to a stateless REST API over JSON, backed by a relational database. A WebSocket channel runs alongside the REST API for the one feature that genuinely needs push delivery — the emergency siren — everything else is request/response.

Client

React 18 SPA (Vite) — role-aware routing, Tailwind CSS styling, Axios for REST, socket.io-client for the live emergency channel.

↓ HTTPS • JSON • Bearer JWT | WebSocket

API server

Node.js + Express 5 — route → middleware (auth, role) → controller → model, one Express Router per domain area.

↓ Sequelize ORM

Database

MySQL / MariaDB — 21 tables, foreign-key relations managed by Sequelize associations; two tables (`audit_logs`) carry DB-level trigger enforcement independent of the application layer.

Real-time delivery is a thin addition to this stack rather than a parallel system: the same Express `app` is wrapped in a raw `http.Server`, and Socket.io attaches to that server so REST and WebSocket traffic share one process and one port.

2 Technology Stack

FRONTEND

- React 18 + Vite
- React Router 6 (role-gated routes)
- Tailwind CSS
- Axios (REST client, interceptor-based auth header)
- socket.io-client (emergency broadcast)
- jsPDF (client-side receipt generation)
- jsQR + qrcode.react (gate-pass QR encode/decode)
- Framer Motion (page/element transitions)

BACKEND

- Node.js + Express 5 (ES modules)
- Sequelize ORM over MySQL2 driver
- MySQL / MariaDB (XAMPP-compatible)
- jsonwebtoken (auth tokens)
- bcryptjs (password hashing)
- Socket.io (emergency broadcast)
- Nodemailer (OTP + reset email, auto Ethereum fallback in dev)
- Multer + Cloudinary (complaint photo uploads, local-disk fallback)

The database choice moved from an original MongoDB/Mongoose plan to MySQL/Sequelize: a housing society's core data — flats, bills, gate logs, complaints — is inherently relational (a bill belongs to exactly one flat, a flat belongs to exactly one block), which a foreign-key-enforced schema models more honestly than a document store, and it matches the SQL skill set the SRS targets.

3 Module Breakdown

The backend is organized by domain, not by technical layer, so a change to one feature touches one controller and one route file rather than being scattered.

MODULE	CONTROLLER	RESPONSIBILITY
Auth	<code>authController</code>	Register, two-step OTP login, forgot/reset password
Admin	<code>adminController</code>	Resident/staff onboarding, billing engine, complaint routing, audit log, dashboard aggregates
Resident	<code>residentController</code>	Own bills, complaints, visitor passes, dashboard
Security	<code>securityController</code> , <code>gateLogController</code>	Gate-pass verification, visitor entry/exit, security alerts
Amenities	<code>amenityController</code>	Amenity catalog, booking, availability
Polls & Notices	<code>pollController</code>	Society polls, voting, notice board
Emergency	<code>emergencyController</code>	Siren trigger/resolve, Socket.io broadcast
Directory	<code>directoryController</code>	Emergency contacts, service staff, guidelines
Family	<code>familyController</code>	Family/tenant member records per flat
Vehicles	<code>vehicleController</code>	Resident vehicle registration
Profile	<code>profileController</code>	Cross-role profile view/edit

The frontend mirrors this by role, not by module: `components/resident`, `components/admin`, and `components/guard` each hold that role's screens, with `components/common` and `components/ui` holding what every role shares (layout chrome, buttons, tables, modals).

4 Security Design

Authentication

Login is two-phase, not one. `POST /auth/login` checks the password against the bcrypt hash and, if correct, emails a 6-digit one-time code — it does not issue a token. `POST /auth/verify-otp` is the only endpoint that ever mints a JWT, and only after the code is confirmed. Codes are single-use, expire after 5 minutes, and re-requesting a code invalidates any earlier unconsumed one. Password reset follows the identical two-artifact pattern with a 30-minute token instead of a 6-digit code.

Authorization

Every protected route passes through `authMiddleware` (verifies the JWT, attaches the decoded user to `req`) and, where the action is role-specific, `roleMiddleware([...roles])`. Role checks are applied per-route rather than blanket-mounted on a router, after an earlier bug where a blanket `router.use(roleMiddleware(["Resident"]))` accidentally shadowed an admin-only route declared after it — the fix was converting to explicit per-route middleware so route order can never silently change who is authorized.

Data integrity

Three categories of activity are journaled to an `audit_logs` table at the moment they happen — gate entries, complaint status changes, and admin financial edits — capturing actor, before/after state, and a timestamp. The table is protected by `BEFORE UPDATE` and `BEFORE DELETE` triggers that raise a SQL exception on any attempt to modify or remove a row, so the guarantee holds even against a direct database connection, not just against the application's own code path.

Transport & secrets

- Passwords are never stored or logged in plaintext — bcrypt hash only.
- JWTs are short-lived and carry role + identity claims only, no sensitive data.
- Database credentials, JWT secret, and mail credentials are read from environment variables, never committed to source.
- The Socket.io emergency channel is intentionally broadcast-only with no per-socket auth — anyone connected receives the alert, matching a physical siren's

reach — while the REST endpoints that trigger or resolve an alert remain JWT- and role-protected.

5 API Summary

Representative endpoints by area; full parameter and response shapes are implemented directly in each controller. All paths are prefixed `/api`.

METHOD	PATH	PURPOSE
POST	<code>/auth/login</code>	Validate credentials, email OTP
POST	<code>/auth/verify-otp</code>	Confirm OTP, issue JWT
POST	<code>/auth/forgot-password</code>	Email reset link
POST	<code>/auth/reset-password</code>	Consume token, set new password
GET	<code>/resident/bills</code>	Own maintenance bills + breakdown
POST	<code>/resident/visitor-pass</code>	Pre-approve a visitor, generate QR
POST	<code>/resident/complaints</code>	Raise a maintenance complaint
POST	<code>/security/verify-pass</code>	Verify a gate-pass code / QR at the gate
POST	<code>/security/log-entry</code>	Log a walk-in visitor, audited
PUT	<code>/admin/complaints/:id/assign</code>	Route ticket to staff, compute SLA due date
POST	<code>/admin/bills/generate</code>	Run the billing engine for a period
PUT	<code>/admin/residents/:id/username</code>	Edit resident account username
POST	<code>/admin/residents/:id/reset-password</code>	Issue a fresh temporary password
GET	<code>/admin/audit-logs</code>	Query the immutable audit trail
POST	<code>/emergency/trigger</code>	Raise the siren, broadcast via Socket.io
PATCH	<code>/emergency/:id/resolve</code>	Clear an active emergency
GET	<code>/family</code>	List family/tenant members for the caller's flat
GET	<code>/directory/service-staff</code>	List on-site trade staff (no login)

6 Role Model

RESIDENT

Bills, complaints, visitor passes, amenity bookings, polls, family/tenant records, emergency siren trigger.

ADMIN

Everything Resident-facing across all flats, plus billing engine, complaint routing/SLA, staff management, audit log, emergency resolution.

GUARD

Gate-pass verification, visitor entry/exit logging, security alerts, overstay monitoring.

A fourth category, **service staff** (plumber, electrician, and similar trades), is modeled deliberately without login credentials — they are a directory entry an admin maintains, not a portal user, since they never need to authenticate into the system.